DEEP LEARNING LAB

Experiments 6, 7, 8 and 9

Corrected Full Python Code, Sample Output and Results

Included experiments

- Experiment 6 — Parts of Speech Tagging using Sequence-to-Sequence Architecture
- Experiment 7 — Machine Translation using Encoder-Decoder Model
- Experiment 8 — Image Augmentation using Generative Adversarial Networks (GANs)
- Experiment 9 — Mini-Project on Real-World Applications: Task Manager

The code is cleaned up to fix formatting and common runtime issues. Neural-network output and loss values can vary by TensorFlow version, hardware, and training run. The POS-tagging and translation examples use tiny demonstration datasets, so they illustrate the architecture rather than provide production-level accuracy.

Experiment 6: Parts of Speech Tagging using Sequence-to-Sequence Architecture

AIM

To implement Parts of Speech (POS) tagging using a sequence-to-sequence encoder-decoder architecture.

Concept

The encoder LSTM processes the input sentence and passes its states to the decoder LSTM. The decoder predicts a POS tag sequence. Start-of-sequence and end-of-sequence tokens support step-by-step inference.

Full Corrected Program

```
# EXPERIMENT 6: PARTS OF SPEECH TAGGING USING SEQUENCE-TO-SEQUENCE
import numpy as np
import tensorflow as tf
from tensorflow.keras.layers import Input, Embedding, LSTM, Dense
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.sequence import pad_sequences

tf.keras.utils.set_random_seed(42)

# Small demonstration dataset (word-level POS tags)
input_texts = [
    "I love coding",
    "This is a pen",
    "She sings well",
]
target_texts = [
    "PRP VB VBG",
    "DT VBZ DT NN",
    "PRP VBZ RB",
]

# Reserve ID 0 for padding. Input and output vocabularies are separate.
input_words = sorted({w for sentence in input_texts for w in sentence.split()})
target_tags = sorted({tag for sentence in target_texts for tag in sentence.split()})

input_word2idx = {word: i + 1 for i, word in enumerate(input_words)}
input_idx2word = {i: word for word, i in input_word2idx.items()}

target_word2idx = {tag: i + 1 for i, tag in enumerate(target_tags)}
target_word2idx["<sos>"] = len(target_word2idx) + 1
target_word2idx["<eos>"] = len(target_word2idx) + 1
target_idx2word = {i: word for word, i in target_word2idx.items()}

SOS_ID = target_word2idx["<sos>"]
EOS_ID = target_word2idx["<eos>"]
INPUT_VOCAB_SIZE = len(input_word2idx) + 1
TARGET_VOCAB_SIZE = len(target_word2idx) + 1
MAX_ENCODER_LEN = max(len(s.split()) for s in input_texts)
MAX_DECODER_LEN = max(len(s.split()) for s in target_texts) + 1  # includes EOS

# Encoder input sequences
encoder_sequences = [
    [input_word2idx[w] for w in sentence.split()]
    for sentence in input_texts
]
encoder_input_data = pad_sequences(
    encoder_sequences, maxlen=MAX_ENCODER_LEN, padding="post"
)

# Decoder input: <sos> + tags; target: tags + <eos>
decoder_input_sequences = []
decoder_target_sequences = []
for sentence in target_texts:
    tag_ids = [target_word2idx[t] for t in sentence.split()]
    decoder_input_sequences.append([SOS_ID] + tag_ids)
    decoder_target_sequences.append(tag_ids + [EOS_ID])

decoder_input_data = pad_sequences(
    decoder_input_sequences, maxlen=MAX_DECODER_LEN, padding="post"
)
decoder_target_data = pad_sequences(
    decoder_target_sequences, maxlen=MAX_DECODER_LEN, padding="post"
)
# Ignore padding positions when calculating loss.
sample_weights = (decoder_target_data != 0).astype("float32")

# Encoder
encoder_inputs = Input(shape=(None,), dtype="int32", name="encoder_inputs")
encoder_embedding_layer = Embedding(
    INPUT_VOCAB_SIZE, 64, mask_zero=True, name="encoder_embedding"
)
encoder_embedded = encoder_embedding_layer(encoder_inputs)
encoder_lstm = LSTM(64, return_state=True, name="encoder_lstm")
_, state_h, state_c = encoder_lstm(encoder_embedded)
encoder_states = [state_h, state_c]

# Decoder
decoder_inputs = Input(shape=(None,), dtype="int32", name="decoder_inputs")
decoder_embedding_layer = Embedding(
```

```
    TARGET_VOCAB_SIZE, 64, mask_zero=True, name="decoder_embedding"
)
decoder_embedded = decoder_embedding_layer(decoder_inputs)
decoder_lstm = LSTM(
    64, return_sequences=True, return_state=True, name="decoder_lstm"
)
decoder_outputs, _, _ = decoder_lstm(
    decoder_embedded, initial_state=encoder_states
)
decoder_dense = Dense(TARGET_VOCAB_SIZE, activation="softmax", name="tag_output")
decoder_outputs = decoder_dense(decoder_outputs)

model = Model([encoder_inputs, decoder_inputs], decoder_outputs)
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    weighted_metrics=["sparse_categorical_accuracy"],
)
model.summary()

print("\nTraining POS tagger...")
model.fit(
    [encoder_input_data, decoder_input_data],
    decoder_target_data,
    sample_weight=sample_weights,
    batch_size=1,
    epochs=250,
    verbose=0,
)
print("Training completed.")

# Inference models
encoder_model = Model(encoder_inputs, encoder_states)
state_input_h = Input(shape=(64,))
state_input_c = Input(shape=(64,))
inference_states = [state_input_h, state_input_c]
inference_decoder_embedded = decoder_embedding_layer(decoder_inputs)
inference_outputs, out_h, out_c = decoder_lstm(
    inference_decoder_embedded, initial_state=inference_states
)
inference_outputs = decoder_dense(inference_outputs)
decoder_model = Model(
    [decoder_inputs] + inference_states,
    [inference_outputs, out_h, out_c],
)

def generate_pos_tags(sentence):
    words = sentence.split()
    ids = [input_word2idx.get(w, 0) for w in words]
    seq = pad_sequences([ids], maxlen=MAX_ENCODER_LEN, padding="post")
    h, c = encoder_model.predict(seq, verbose=0)
    current_token = np.array([[SOS_ID]], dtype=np.int32)
    predicted_tags = []

    for _ in range(MAX_DECODER_LEN):
        probabilities, h, c = decoder_model.predict(
            [current_token, h, c], verbose=0
        )
        token_id = int(np.argmax(probabilities[0, -1, :]))
        if token_id == EOS_ID or token_id == 0:
            break
        predicted_tags.append(target_idx2word.get(token_id, "UNK"))
        current_token = np.array([[token_id]], dtype=np.int32)

    return " ".join(predicted_tags)

print("\nPOS Tagging Results:")
for sentence in input_texts:
    print("Input:", sentence)
    print("Predicted POS Tags:", generate_pos_tags(sentence))
    print()

print("Note: This is a tiny demonstration dataset; use a larger tagged corpus")
print("for reliable POS tagging on unseen sentences.")
```

Sample Output

```
Training POS tagger...
Training completed.

POS Tagging Results:
Input: I love coding
Predicted POS Tags: (model-generated tags)

Input: This is a pen
Predicted POS Tags: (model-generated tags)

Input: She sings well
Predicted POS Tags: (model-generated tags)

Note: This is a tiny demonstration dataset; use a larger tagged corpus
for reliable POS tagging on unseen sentences.
```

Result

Thus, Parts of Speech Tagging using a Sequence-to-Sequence architecture is implemented.

Note

The exact predicted tags can vary because the training dataset contains only three examples. The program prints the model's actual predictions when executed.

Experiment 7: Machine Translation using Encoder-Decoder Model

AIM

To implement a simple machine translation model using an encoder-decoder LSTM architecture.

Concept

The encoder reads source-language tokens and produces hidden states. The decoder learns to generate target-language tokens using teacher forcing during training and autoregressive decoding during inference.

Full Corrected Program

```
# EXPERIMENT 7: MACHINE TRANSLATION USING ENCODER-DECODER
import numpy as np
import tensorflow as tf
from tensorflow.keras.layers import Input, Embedding, LSTM, Dense
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.sequence import pad_sequences

tf.keras.utils.set_random_seed(42)

# Small English-to-German demonstration dataset
input_texts = [
    "I love coding",
    "This is a pen",
    "She sings well",
]
target_texts = [
    "Ich liebe das Coden",
    "Das ist ein Stift",
    "Sie singt gut",
]

# Word-level vocabularies; ID 0 is reserved for padding.
input_words = sorted({w for s in input_texts for w in s.split()})
target_words = sorted({w for s in target_texts for w in s.split()})

input_word2idx = {w: i + 1 for i, w in enumerate(input_words)}
target_word2idx = {w: i + 1 for i, w in enumerate(target_words)}
target_word2idx["<sos>"] = len(target_word2idx) + 1
target_word2idx["<eos>"] = len(target_word2idx) + 1
target_idx2word = {i: w for w, i in target_word2idx.items()}

SOS_ID = target_word2idx["<sos>"]
EOS_ID = target_word2idx["<eos>"]
INPUT_VOCAB_SIZE = len(input_word2idx) + 1
TARGET_VOCAB_SIZE = len(target_word2idx) + 1
MAX_ENCODER_LEN = max(len(s.split()) for s in input_texts)
MAX_DECODER_LEN = max(len(s.split()) for s in target_texts) + 1

encoder_sequences = [
    [input_word2idx[w] for w in sentence.split()]
    for sentence in input_texts
]
encoder_input_data = pad_sequences(
    encoder_sequences, maxlen=MAX_ENCODER_LEN, padding="post"
)

# Teacher forcing: decoder sees <sos> + target words;
# expected output is target words + <eos>.
decoder_input_sequences = []
decoder_target_sequences = []
for sentence in target_texts:
    ids = [target_word2idx[w] for w in sentence.split()]
    decoder_input_sequences.append([SOS_ID] + ids)
    decoder_target_sequences.append(ids + [EOS_ID])

decoder_input_data = pad_sequences(
    decoder_input_sequences, maxlen=MAX_DECODER_LEN, padding="post"
)
decoder_target_data = pad_sequences(
    decoder_target_sequences, maxlen=MAX_DECODER_LEN, padding="post"
)
sample_weights = (decoder_target_data != 0).astype("float32")

# Encoder
encoder_inputs = Input(shape=(None,), dtype="int32", name="encoder_inputs")
encoder_embedding_layer = Embedding(
    INPUT_VOCAB_SIZE, 64, mask_zero=True, name="encoder_embedding"
)
encoder_embedded = encoder_embedding_layer(encoder_inputs)
encoder_lstm = LSTM(64, return_state=True, name="encoder_lstm")
_, state_h, state_c = encoder_lstm(encoder_embedded)
encoder_states = [state_h, state_c]

# Decoder
decoder_inputs = Input(shape=(None,), dtype="int32", name="decoder_inputs")
decoder_embedding_layer = Embedding(
    TARGET_VOCAB_SIZE, 64, mask_zero=True, name="decoder_embedding"
)
decoder_embedded = decoder_embedding_layer(decoder_inputs)
```

```
decoder_lstm = LSTM(
    64, return_sequences=True, return_state=True, name="decoder_lstm"
)
decoder_outputs, _, _ = decoder_lstm(
    decoder_embedded, initial_state=encoder_states
)
decoder_dense = Dense(TARGET_VOCAB_SIZE, activation="softmax", name="word_output")
decoder_outputs = decoder_dense(decoder_outputs)

model = Model([encoder_inputs, decoder_inputs], decoder_outputs)
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    weighted_metrics=["sparse_categorical_accuracy"],
)
model.summary()

print("\nTraining translation model...")
model.fit(
    [encoder_input_data, decoder_input_data],
    decoder_target_data,
    sample_weight=sample_weights,
    batch_size=1,
    epochs=300,
    verbose=0,
)
print("Training completed.")

# Inference encoder and decoder
encoder_model = Model(encoder_inputs, encoder_states)
state_input_h = Input(shape=(64,))
state_input_c = Input(shape=(64,))
inference_states = [state_input_h, state_input_c]
inference_embedded = decoder_embedding_layer(decoder_inputs)
inference_outputs, out_h, out_c = decoder_lstm(
    inference_embedded, initial_state=inference_states
)
inference_outputs = decoder_dense(inference_outputs)
decoder_model = Model(
    [decoder_inputs] + inference_states,
    [inference_outputs, out_h, out_c],
)

def translate(sentence):
    words = sentence.split()
    ids = [input_word2idx.get(w, 0) for w in words]
    seq = pad_sequences([ids], maxlen=MAX_ENCODER_LEN, padding="post")
    h, c = encoder_model.predict(seq, verbose=0)
    current_token = np.array([[SOS_ID]], dtype=np.int32)
    translated_words = []

    for _ in range(MAX_DECODER_LEN):
        probabilities, h, c = decoder_model.predict(
            [current_token, h, c], verbose=0
        )
        token_id = int(np.argmax(probabilities[0, -1, :]))
        if token_id == EOS_ID or token_id == 0:
            break
        translated_words.append(target_idx2word.get(token_id, "UNK"))
        current_token = np.array([[token_id]], dtype=np.int32)

    return " ".join(translated_words)

print("\nTranslation Results:")
for sentence in input_texts:
    print("Input:", sentence)
    print("Translated Text:", translate(sentence))
    print()

print("Note: Three sentence pairs are only for demonstrating the model pipeline.")
print("Real translation requires a large parallel corpus and more training.")
```

Sample Output

```
Training translation model...
Training completed.

Translation Results:
Input: I love coding
Translated Text: (model-generated German words)

Input: This is a pen
Translated Text: (model-generated German words)

Input: She sings well
Translated Text: (model-generated German words)

Note: Three sentence pairs are only for demonstrating the model pipeline.
Real translation requires a large parallel corpus and more training.
```

Result

Thus, Machine Translation using an Encoder-Decoder model is implemented.

Note

The output text is generated by the trained model and may not be a correct translation. Three sentence pairs are insufficient to train a reliable translation system.

Experiment 8: Image Augmentation using Generative Adversarial Networks (GANs)

AIM

To implement a GAN that learns from MNIST images and generates new handwritten-like digit images.

Concept

A GAN contains a generator that creates images from random noise and a discriminator that distinguishes real images from generated images. Both networks are trained in an adversarial process.

Full Corrected Program

```
# EXPERIMENT 8: IMAGE AUGMENTATION USING GANs
import os
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras import layers

tf.keras.utils.set_random_seed(42)

# 1. Load MNIST and use a subset for a shorter lab run.
(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
x_train = x_train[:12000].astype("float32")

# Normalize pixels from [0, 255] to [-1, 1]; add grayscale channel.
x_train = (x_train - 127.5) / 127.5
x_train = np.expand_dims(x_train, axis=-1)
print("Dataset shape:", x_train.shape)

BATCH_SIZE = 128
NOISE_DIM = 100
EPOCHS = 10

train_dataset = (
    tf.data.Dataset.from_tensor_slices(x_train)
    .shuffle(len(x_train))
    .batch(BATCH_SIZE, drop_remainder=True)
    .prefetch(tf.data.AUTOTUNE)
)

# 2. Generator: random noise -> 28x28 grayscale image.
def build_generator():
    return tf.keras.Sequential([
        layers.Input(shape=(NOISE_DIM,)),
        layers.Dense(7 * 7 * 128, use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Reshape((7, 7, 128)),
        layers.Conv2DTranspose(
            64, kernel_size=4, strides=2, padding="same", use_bias=False
        ),
        layers.BatchNormalization(),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Conv2DTranspose(
            1, kernel_size=4, strides=2, padding="same", activation="tanh"
        ),
    ], name="Generator")

# 3. Discriminator: image -> real/fake logit.
def build_discriminator():
    return tf.keras.Sequential([
        layers.Input(shape=(28, 28, 1)),
        layers.Conv2D(64, kernel_size=4, strides=2, padding="same"),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Dropout(0.3),
        layers.Conv2D(128, kernel_size=4, strides=2, padding="same"),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Dropout(0.3),
        layers.Flatten(),
        layers.Dense(1),
    ], name="Discriminator")

generator = build_generator()
discriminator = build_discriminator()
print("\nGenerator Summary:")
generator.summary()
print("\nDiscriminator Summary:")
discriminator.summary()

# 4. Losses and optimizers.
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)
generator_optimizer = tf.keras.optimizers.Adam(0.0002, beta_1=0.5)
discriminator_optimizer = tf.keras.optimizers.Adam(0.0002, beta_1=0.5)

# 5. One training step: update both networks.
@tf.function
def train_step(real_images):
    noise = tf.random.normal([BATCH_SIZE, NOISE_DIM])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
```

```
        fake_images = generator(noise, training=True)
        real_output = discriminator(real_images, training=True)
        fake_output = discriminator(fake_images, training=True)

        gen_loss = cross_entropy(tf.ones_like(fake_output), fake_output)
        real_loss = cross_entropy(tf.ones_like(real_output), real_output)
        fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
        disc_loss = real_loss + fake_loss

    gen_gradients = gen_tape.gradient(
        gen_loss, generator.trainable_variables
    )
    disc_gradients = disc_tape.gradient(
        disc_loss, discriminator.trainable_variables
    )
    generator_optimizer.apply_gradients(
        zip(gen_gradients, generator.trainable_variables)
    )
    discriminator_optimizer.apply_gradients(
        zip(disc_gradients, discriminator.trainable_variables)
    )
    return gen_loss, disc_loss

# 6. Train the GAN.
for epoch in range(EPOCHS):
    gen_total = 0.0
    disc_total = 0.0
    batches = 0

    for real_batch in train_dataset:
        gen_loss, disc_loss = train_step(real_batch)
        gen_total += float(gen_loss)
        disc_total += float(disc_loss)
        batches += 1

    print(
        f"Epoch {epoch + 1}/{EPOCHS} | "
        f"Generator Loss: {gen_total / batches:.4f} | "
        f"Discriminator Loss: {disc_total / batches:.4f}"
    )

print("\nGAN training completed!")

# 7. Generate, display, and save 16 images.
os.makedirs("gan_outputs", exist_ok=True)
noise = tf.random.normal([16, NOISE_DIM])
generated_images = generator(noise, training=False)

plt.figure(figsize=(8, 8))
for i in range(16):
    image = generated_images[i, :, :, 0].numpy()
    image = np.clip((image + 1.0) * 127.5, 0, 255).astype(np.uint8)

    plt.subplot(4, 4, i + 1)
    plt.imshow(image, cmap="gray", vmin=0, vmax=255)
    plt.axis("off")

    # IMPORTANT: save_img requires rank-3 shape (height, width, channels).
    image_to_save = np.expand_dims(image, axis=-1)
    tf.keras.utils.save_img(
        f"gan_outputs/generated_image_{i + 1}.png",
        image_to_save,
        scale=False,
    )

plt.tight_layout()
plt.savefig("gan_outputs/generated_images_grid.png", bbox_inches="tight")
plt.show()

generator.save("gan_outputs/gan_generator.keras")
print("Saved 16 generated images in gan_outputs/")
print("Saved grid: gan_outputs/generated_images_grid.png")
print("Saved generator: gan_outputs/gan_generator.keras")
```

Sample Output

```
Dataset shape: (12000, 28, 28, 1)

Generator Summary:
Model: Generator
Output shape: (None, 28, 28, 1)

Discriminator Summary:
Model: Discriminator
Output shape: (None, 1)

Epoch 1/10 | Generator Loss: <value> | Discriminator Loss: <value>
...
Epoch 10/10 | Generator Loss: <value> | Discriminator Loss: <value>

GAN training completed!
[A 4 x 4 grid of 16 generated images is displayed]
Saved 16 generated images in gan_outputs/
Saved grid: gan_outputs/generated_images_grid.png
Saved generator: gan_outputs/gan_generator.keras
```

Result

Thus, Image Augmentation using GANs is implemented successfully.

Note

Loss values and generated image quality vary between runs. The image-saving code explicitly adds the grayscale channel dimension, preventing the earlier save_img rank error.

Experiment 9: Mini-Project on Real-World Applications: Task Manager

AIM

To develop a simple task manager that allows users to add tasks, mark tasks as completed, view tasks, and view a summary.

Concept

The Python program stores task names, statuses, and dates in tasks.txt. It loads saved records when started and updates the file when tasks are added, completed, or when the program exits.

Full Corrected Program

```
# EXPERIMENT 9: MINI-PROJECT - TASK MANAGER
import os
import datetime

# Read saved tasks from a text file.
def load_tasks(file_path):
    tasks = []
    if os.path.exists(file_path):
        with open(file_path, "r", encoding="utf-8") as file:
            for line in file:
                line = line.strip()
                if not line:
                    continue
                try:
                    task_name, status, date_str = line.split(",", 2)
                    task_date = (
                        datetime.datetime.strptime(date_str, "%Y-%m-%d").date()
                        if date_str else None
                    )
                    tasks.append({
                        "task": task_name,
                        "status": status,
                        "date": task_date,
                    })
                except ValueError:
                    print("Warning: skipping an invalid task record.")
    return tasks

# Save tasks to a text file.
def save_tasks(file_path, tasks):
    with open(file_path, "w", encoding="utf-8") as file:
        for task in tasks:
            date_str = (
                task["date"].strftime("%Y-%m-%d")
                if task["date"] else ""
            )
            # Commas in task names are replaced when adding tasks.
            file.write(f'{task["task"]},{task["status"]},{date_str}\n')

# Add a new task.
def add_task(tasks, new_task):
    tasks.append({
        "task": new_task.replace(",", ";"),
        "status": "Incomplete",
        "date": datetime.date.today(),
    })
    print(f"Task '{new_task}' added successfully.")

# Mark a task as completed.
def complete_task(tasks, task_index):
    if 1 <= task_index <= len(tasks):
        tasks[task_index - 1]["status"] = "Completed"
        tasks[task_index - 1]["date"] = datetime.date.today()
        print(f"Task '{tasks[task_index - 1]['task']}' marked as completed.")
    else:
        print("Invalid task index.")

# Display all tasks.
def view_tasks(tasks):
    if not tasks:
        print("No tasks found.")
        return
    print("\nTasks:")
    for i, task in enumerate(tasks, start=1):
        date_str = task["date"].strftime("%Y-%m-%d") if task["date"] else "No date"
        print(f"{i}. {task['task']} - {task['status']} ({date_str})")

# Display task counts.
def view_summary(tasks):
    completed = sum(task["status"] == "Completed" for task in tasks)
    incomplete = sum(task["status"] == "Incomplete" for task in tasks)
    print("\nSummary:")
    print(f"Total tasks: {len(tasks)}")
    print(f"Completed tasks: {completed}")
    print(f"Incomplete tasks: {incomplete}")

def main():
```

```
    file_path = "tasks.txt"
    tasks = load_tasks(file_path)

    while True:
        print("\n===== TASK MANAGER =====")
        print("1. Add Task")
        print("2. Complete Task")
        print("3. View Tasks")
        print("4. View Summary")
        print("5. Quit")
        choice = input("Enter your choice (1/2/3/4/5): ").strip()

        if choice == "1":
            new_task = input("Enter the task: ").strip()
            if new_task:
                add_task(tasks, new_task)
                save_tasks(file_path, tasks)
            else:
                print("Task cannot be empty.")
        elif choice == "2":
            view_tasks(tasks)
            if tasks:
                try:
                    index = int(input("Enter the task number to mark as completed: "))
                    complete_task(tasks, index)
                    save_tasks(file_path, tasks)
                except ValueError:
                    print("Please enter a valid integer.")
        elif choice == "3":
            view_tasks(tasks)
        elif choice == "4":
            view_summary(tasks)
        elif choice == "5":
            save_tasks(file_path, tasks)
            print("Exiting. Tasks saved.")
            break
        else:
            print("Invalid choice. Please select 1 to 5.")

if __name__ == "__main__":
    main()
```

Sample Output

```
===== TASK MANAGER =====
1. Add Task
2. Complete Task
3. View Tasks
4. View Summary
5. Quit
Enter your choice (1/2/3/4/5): 1
Enter the task: Complete Python mini-project
Task 'Complete Python mini-project' added successfully.

Enter your choice (1/2/3/4/5): 3
Tasks:
1. Complete Python mini-project - Incomplete (YYYY-MM-DD)

Enter your choice (1/2/3/4/5): 4
Summary:
Total tasks: 1
Completed tasks: 0
Incomplete tasks: 1

Enter your choice (1/2/3/4/5): 5
Exiting. Tasks saved.
```

Result

Thus, the Mini-Project on Real-World Applications (Task Manager) is implemented successfully using Python.

Note

Run this program in VS Code, IDLE, or a Python terminal. The tasks.txt file is created in the current working directory. In Google Colab, download the file if you want to keep it after the session.